Table of Contents

1. System Overview and Repository Baseline 1

2. Architecture 2

3. Data Model and Migrations 4

4. Multi-Tenancy, Authentication and Security 6

5. Job System and Render Pipeline 8

6. API Surface (Route Map) 9

7. AI Providers and Intelligent Engines 10

8. Testing and Quality Assurance 12

9. CI/CD 14

10. Deployment and Operations 15

11. Documentation Audit and Maintenance Runbook 16

Appendix A. Quick Reference 19

1. System Overview and Repository Baseline

TubeForge is a multi-tenant AI video production platform. A user inside an organization moves from a script idea to a published video through AI-assisted scripting, storyboarding, casting, voiceover, rendering and publishing, all inside one product. The front end is a Next.js 16 App Router application (Zustand for state, TanStack Query for data); the back end is a Prisma-managed PostgreSQL database, a default-deny API surface of 123 route files, and an independent worker process that consumes a database-backed durable queue and runs the render pipeline. One Docker image serves both the web and worker processes in production.

Repository state at the time of writing: **origin/main is ce7e9c3** with CI green on all six jobs and the unit suite at 471/471. Local HEAD is **d03bed9**, two commits ahead of origin and not yet pushed (the development sandbox holds no TF_TOKEN credential). The delta is the boot-payload code-splitting fix described in Section 2.2 and Chapter 9. Every figure in this handbook was re-measured against the working tree at d03bed9 on 2026-11-12; where a repository document disagrees, Chapter 11 records the drift explicitly.

Metric	Value at d03bed9
Source files under src/	356
Library modules (src/lib/tubeforge)	132
Provider layer files	11
API route files (src/app/api, route.ts)	123
Prisma models (schema.prisma)	32
Prisma migrations	23
Unit tests passing	471 / 471
Playwright e2e specifications	7
CI jobs (ci.yml) + nightly battery	6 + 1
Frontend views	11 (1 eager + 10 lazy)
Worklog session headers	67 (S0 through S94)
ROADMAP 2.0 items landed at v1	13 / 13

Table 1 - Repository baseline, re-measured 2026-11-12.

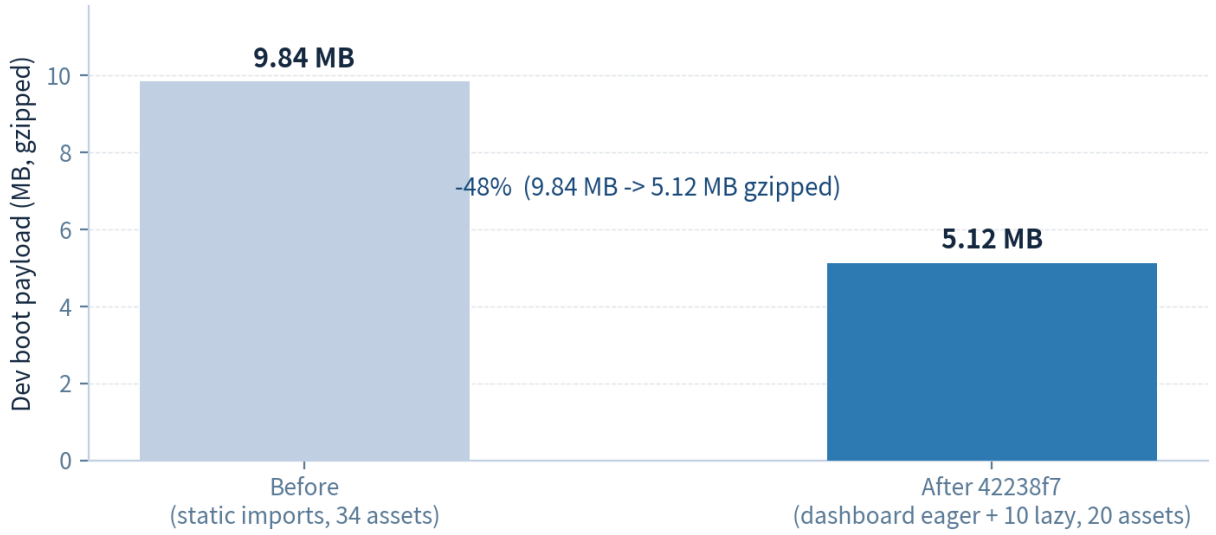


Figure 1 - Dev boot payload before and after the code-splitting fix (commit d03bed9), as measured by scripts/measure-boot.js.

How to read this handbook. Chapters 2 through 10 describe the system as it exists: architecture, data model, security, the job and render pipeline, the API surface, the provider matrix, testing, CI/CD, and deployment. Chapter 11 is the audit chapter: it lists every place where the repository documentation has drifted behind the code, together with the maintenance runbook and the standing risk register. Appendix A collects the commands, environment variables and document index a maintainer needs day to day.

2. Architecture

2.1 Topology

The browser talks HTTPS to a Next.js 16 standalone server (Node 20), authenticating with either a session cookie or a scoped bearer API token. Inside the server, src/middleware.ts is a default-deny gatekeeper whose public allow-list contains exactly six auth endpoints plus the YouTube OAuth callback; everything else requires an authenticated, tenant-scoped request. The 123 API route files under src/app/api are tenant-scoped through async-local-storage (ALS): tenant-db.ts, a Prisma $extends client, injects the organizationId of the requesting membership into every query. The 132 modules of src/lib/tubeforge carry the domain logic, including the 11 provider files that wrap external AI services. A separate worker process (scripts/worker.mts) consumes the durable job queue in Postgres; Compose and Fly run web and worker from the same image.

Downstream of the app sit five dependencies: PostgreSQL (Prisma with versioned migrations), local disk or S3/R2-compatible object storage with presigned-URL delivery, the AI provider gateway (chat, TTS, image, video, ASR, vision), the YouTube Data API with OAuth for publishing and analytics, and the stock footage sources Pexels and Pixabay. The worker additionally needs ffmpeg and, for the whiteboard and 3D lanes, a headless Chromium (Chapter 5).

2.2 Frontend: eleven views, one eager

The application surface is composed of eleven views: dashboard, create, library, cast, voice, research, channels, calendar, analytics, settings and world. Until revision d03bed9 the landing page statically imported all eleven, so a phone on a 4G connection could sit on the AuthGate splash ("Warming up the forge...") for minutes while roughly 10 MB of gzipped JavaScript arrived. The fix keeps the dashboard eager and streams the other ten views in on first visit; the loaders unwrap the named exports because the view files carry no default export. The dev boot payload fell from 9.84 MB to 5.12 MB gzipped across 34 to 20 assets (Figure 1), and scripts/measure-boot.js was added as the standing boot-payload diagnostic.

View	What it owns
dashboard	Mission Control - the landing overview a user meets after the gate.
create	The script-to-video creation flow.
library	The video library and delivered cuts.
cast	Characters, casting and appearance identity.
voice	Voice profiles and voiceover synthesis.
research	Research sessions and source material.
channels	YouTube channels, publishing and analytics.
calendar	The scheduling surface (ScheduleSlot).
analytics	Video statistics and performance.
settings	Account and organization settings, provider credentials, AI scoring.
world	The world bible - locations and props (Stage 5).

Table 2 - The eleven frontend views under src/components/tubeforge/views.

2.3 Process model and shared state

Web and worker are stateless relative to each other: all shared state lives in PostgreSQL (the job queue, the JSON document columns, usage and reservation records) and in object storage. That is what lets the Fly deployment scale the worker independently of the web tier, and what makes a crashed job resumable - the worker re-reads its payload from the database and continues deterministically. The optional voice server is the one component that wants a persistent box; it is addressed over HTTP and never shares memory with the app (Chapter 10).

2.4 Current versus target architecture

ARCHITECTURE.md keeps two pictures: Section 1 is the current architecture, reconciled against the live source at S75 (2026-11-10), and Sections 2 through 8 remain the target designs from the S47 era. The reconciliation that produced the current picture is recorded in docs/audits/s75-architecture-reconciliation-record.md. Since S75, the S88 through S94 slices landed without a fresh reconciliation - their design contracts live in docs/design/s84 through docs/design/s94 - which is the root cause of the documentation drift audited in Chapter 11.

3. Data Model and Migrations

3.1 The tenancy spine

The schema (prisma/schema.prisma) declares 32 models. The tenancy spine is User, Organization and Membership, with the four roles owner, admin, editor and viewer. Every content model carries an organizationId column with cascade delete and an index, so a tenant evacuation is one delete and isolation is structural, not conventional. Settings are per-user rows (UserSetting), never singletons, and provider credentials plus YouTube tokens are encrypted per record with AES-256-GCM under a v1. key format (Chapter 4).

3.2 The 32 models at a glance

Family	Models
Tenancy spine	User, Organization, Membership
Access	Session, ApiToken, Setting, UserSetting, AuthAttempt
Integrations	YouTubeAccount, ProviderCredential, Channel
Creative core	Video, Character, CharacterAppearance, WhiteboardAssetUpload, VoiceProfile, MusicTrack, Location, Prop
Operations	Job, Activity, ResearchSession, ScheduleSlot, VideoStat
Billing and usage	StorageUsage, UsageRecord, SpendReservation, Plan, Subscription, CreditGrant, CreditLedger
Integrity	ProcessedWebhookEvent (exactly-once webhook handling)

Table 3 - The 32 Prisma models grouped by family.

3.3 The JSON document columns

The heavier creative state lives in versioned JSON document columns on Video rather than in wide relational tables. The docs follow one law set in the S88 era: versioned on write, tolerant on parse, strict on write, round-trip safe. The timeline document carries a schema stamp (1 or 2) with a guard that refuses to overwrite a document stamped by an unknown future version across all six write paths - the stale-tracks hazard is structurally impossible rather than policed. referenceAnalysis (S88) is the ninth such column and scene3d (S94) the tenth, with its own 3-pin contract: junk, version and empty payloads are refused, the round-trip is byte-stable, and the light law guarantees an ambient default so a scene never renders black.

3.4 The 23 migrations

Twenty-three migrations are on record, one initial plus twenty-two feature migrations. Production ships them with prisma migrate deploy; local development uses the generated SQLite mirror (Section 3.5). The two most recent - s88_reference_analysis and s94_scene3d - postdate the S75 architecture reconciliation, which is why ARCHITECTURE.md still counts 22.

Migration	What it introduced
000000000000_init	The Stage-1 baseline schema (11-model, single-user era).
stage2_multiuser	User / Organization / Membership; organizationId on content models.
stage3_queue	The durable job queue tables.
stage3b_usage_storage	Usage records and storage accounting.
stage3c_job_payload	Versioned job payload columns.
stage5_director / _world / _sources	AI Director plan, world bible and source documents.
stage6_publishing	Publishing and channel targets.
stage7_billing (+ s71, s72_credit_reservations)	Billing, spend integrity, credit reservations.
auth_setting	UserSetting rows (per-user settings).
s20_api_token_registry	The 90-day scoped API token registry.
s32_spend_reservations	Spend-gate reservation ledger.
s40_qc_enforcement	QC enforcement records.
s46_auth_attempt	AuthAttempt table backing login lockout.
s49_content_policy	Per-org content policy.
s67_whiteboard_doc / s69_whiteboard_assets	Whiteboard document and its asset rows.
s72_retention_report	Retention reporting.

Migration	What it introduced
s88_reference_analysis	referenceAnalysis - the ninth JSON doc column (S88).
s94_scene3d	Video.scene3d - the tenth JSON doc column (S94).

Table 4 - The 23 Prisma migrations, grouped by the feature that shipped them.

3.5 The SQLite mirror cannot drift

Production runs PostgreSQL; local development runs SQLite. There is exactly one source schema - prisma/schema.prisma - and the SQLite variant is generated from it by scripts/sync-sqlite-schema.mjs, then provisioned with prisma db push in the setup path. A hand-edited second schema is therefore impossible by construction; schema changes always ship as versioned migrations against the single source. The cost of the mirror is one extra script in the setup chain (npm run setup:local), the benefit is that a laptop developer and the production database agree by construction.

4. Multi-Tenancy, Authentication and Security

4.1 Accounts and sessions

Passwords are stored as PBKDF2 hashes. Sessions are server-side Session rows - seven-day, individually revocable - behind signed cookies, so a logout or a compromise response is a row delete, not a wait for expiry. API access uses 90-day scoped ApiToken bearer tokens from the s20 registry. Email verification and password reset are built in, and GDPR export and delete close the account lifecycle.

4.2 The default-deny gate

src/middleware.ts denies everything by default. The public allow-list is six auth endpoints plus /api/youtube/callback; every other route requires an authenticated session or token. A production fail-safe refuses to serve at all when TUBEFORGE_AUTH_SECRET is missing - the failure mode is a down deployment, never an open one. Login lockout (S46) counts failures in a rolling ten-minute window per account, locks out after five, and persists through the AuthAttempt table; the check runs through a second Prisma client so a migration or a multi-instance roll can never leave it forgotten. The documented availability choice is that a lockout-read error fails open (announced, not silent).

4.3 Tenant scoping in the data layer

The organizationId of the requesting membership is carried in the request context through async local storage. tenant-db.ts exposes a Prisma $extends client that injects that organizationId into queries, and raw imports of

@/lib/db in route code are lint-blocked, so the scoping is not a convention a new endpoint can forget. Cross-tenant isolation is pinned end to end by e2e/isolation.spec.ts and by the per-stage route probes that run in CI.

4.4 Secrets and request hygiene

Secrets at rest - ProviderCredential.apiKey and the YouTube tokens - are AES-256-GCM encrypted per record in a v1. envelope format, so a database dump alone does not yield provider keys. Mutations carry an origin check for CSRF atop SameSite=Lax cookies. Outbound fetches go through net-guard.ts safeFetch, which blocks private, loopback, CGNAT and cloud-metadata targets, re-resolves DNS per redirect hop to defeat rebinding, and caps both bytes and wall time. The S46 lesson generalizes: every security table is verified through its own Prisma client so infrastructure rolls cannot silently drop it.

4.5 The security model at a glance

Mechanism	Implementation	Pinned by
Password hashing	PBKDF2	Unit suite
Sessions	7-day, revocable Session rows behind signed cookies	Route probes
API tokens	90-day scoped ApiToken registry (s20)	Unit suite
The gate	Default-deny middleware; 6 public auth endpoints + YouTube callback	Route battery
Login lockout	5 failures / rolling 10 min, AuthAttempt, second Prisma client	S46 records
Secrets at rest	AES-256-GCM per record, v1. envelope	Crypto unit pins
CSRF	Origin check on mutations + SameSite=Lax cookies	Route probes
SSRF	safeFetch: private / loopback / CGNAT / metadata blocked, DNS re-resolved per hop, byte and time caps	net-guard tests
Tenant scoping	tenant-db $extends ALS injection; raw @/lib/db lint-blocked	e2e isolation spec

Table 5 - The security model, mechanism by mechanism.

5. Job System and Render Pipeline

5.1 The durable queue

Rendering is too slow and too failure-prone to run inside a request. Every long operation - renders, reference analysis, 3D scenes, critic runs - is a row in the Postgres job queue, consumed by scripts/worker.mts. Jobs are crash-resumable by design: the payload is versioned and deterministic re-render on resume is the contract, so a worker restart mid-job costs time, not correctness. The queue ledger lesson from S88 is now standing practice: when a job kind is added, every kind ledger is updated in the same slice, which is how the S94 3D lane shipped both ledgers moving 23 to 24 kinds together.

5.2 The render lanes

Four engines produce footage, all feeding the same timeline composition contract. The compositor lane renders image-backed scenes with kenburns and zoompan motion through ffmpeg. The whiteboard lane rasterizes frame plans in headless Chromium driven by Playwright, screenshotting frame by frame on a frame-quantized clock before ffmpeg assembly. The slides engine keeps its own browser path with the same launch discipline. The 3D lane (S94) renders three.js r187 scenes under WebGL 2.0 on SwiftShader inside the same proven Chromium pipeline - three's relative core import is resolved by dual blob injection - and the resulting clip rides the timeline as an ordinary video-kind asset: the compositor cannot tell it from stock footage, and must not. The character rig (S93) is a motion source, not a fifth pipeline: it emits zoompan-vocabulary plans for portrait-backed cast scenes with a breath cycle capped at three percent amplitude - "a whisper, never a punch" - and sits at the bottom of the per-shot seniority ladder.

5.3 The Chromium worker contract

The production image must ship a browser or the whiteboard and 3D lanes die at chromium.launch(). The contract, measured in the S75 E10 benchmark and the E12 live queue drill: install Chromium in the runner stage with PLAYWRIGHT_BROWSERS_PATH=/opt/ms-playwright and npx playwright install --with-deps chromium (the pinned playwright 1.57.0 resolves browser build chromium-1200 from the image's own node_modules), and launch with --no-sandbox --disable-dev-shm-usage - encoded as WB_CHROMIUM_LAUNCH_ARGS in src/lib/tubeforge/whiteboard/render.ts and used at both launch sites. The measured reason: on a 4 GB worker with the default 64 MB /dev/shm, a bare launch is SIGKILLed mid-scene by the OOM killer. The Dockerfile lines have landed and the dependency closure is verified statically, but the image build itself has NOT been smoke-tested - the sandbox has no Docker daemon - and that smoke remains a required production gate (Chapter 11).

5.4 The Universal Timeline contract (S87)

The timeline document is the spine every engine writes into. S87 closed the Universal Timeline contract at the code level across six slices: the version stamp, the v2 dual-write that derives tracks at serialization (a stale-tracks document is structurally impossible), the composition plan that dispatches per-scene on kind, the whiteboard lane emitting tracks on the one frame-quantized clock, the collapse of the legacy branches, and the acceptance battery. The battery (scripts/s87-acceptance-battery.mts) runs real renders through both engine lanes with zero AI spend, holds 18 of 18 checks, and is wired into the CI e2e job; its first run caught and fixed a real engine bug - a mixed-voiceover assembly crash in the whiteboard lane. One tail is recorded: the slice-4 soak gate, fourteen or more days with zero v1-document renders before the legacy fallback retires.

5.5 The audio contract

The S76 close-out struck the synthetic-audio caveat from the whiteboard smoke and pinned the loudness contract at the engine boundary: loudness is measured and enforced where the audio is mixed, not policed per caller. The whiteboard lane carries the same voiceover interlock as the compositor - a render does not ship without its audio contract satisfied - and both lanes compose on the one frame-quantized clock, so a scene window is by construction the delivered frames for it. The S87 battery caught the last known audio edge: a mixed-voiceover assembly with partial voice coverage previously crashed the whiteboard render, and the fix is pinned by the battery rather than by a unit test alone.

6. API Surface (Route Map)

There are 123 route.ts files under src/app/api at d03bed9 - the S75 reconciliation recorded 116, and the S88 through S94 slices added the rest. The surface is uniform: every route is tenant-scoped through the ALS-injected Prisma client, every mutation carries the origin check, and the middleware gate in front of all of them is default-deny. The table below is the maintenance map - where to look when a surface changes, and what each area owns.

Area	Representative endpoints	Notes
Auth (public)	login, register, verify, reset, session	The 6-endpoint public allow-list; everything else denies.
Account / tokens	GET/POST/DELETE /api/tokens	90-day scoped ApiToken registry (s20).
Videos & scenes	/api/videos, /api/videos/[id]/scenes	Script, storyboard, casting, assembly.
Director & plan	/api/videos/[id]/plan, /director	AI Director plan document and approvals.
Voice & TTS	/api/videos/[id]/voice	Voiceover synthesis and verification.
Render	POST/GET /api/videos/[id]/render	Both engine lanes; kind-ledger dispatch.

Area	Representative endpoints	Notes
Stock	/api/videos/[id]/stock	Pexels + Pixabay adapters, ranker, verifier (S89).
Whiteboard	/api/videos/[id]/whiteboard	Whiteboard document and assets (s67/s69).
Reference (S88)	POST/GET/DELETE /api/videos/[id]/reference	Upload-only ingestion; crash-resumable analysis job.
Scene3D (S94)	POST/GET/DELETE /api/videos/[id]/scene3d	3D contract doc; deterministic re-render on resume.
Board (S91)	GET /api/videos/[id]/board	One composed production snapshot; no new tables.
Pacing (S92)	GET /api/videos/[id]/pacing	Measured words-per-minute strip; advisory only.
AI score (S90)	GET/PUT /api/settings/ai/score	Provider scoring surface; advisory, never dispatch.
YouTube	/api/youtube/* (+ public callback)	OAuth publish and analytics.
Billing & credits	/api/billing, /api/credits	Spend gates, reservations, integrity checks.
Admin & retention	/api/admin/*	QC enforcement, content policy, retention reports.

Table 6 - Route map by area. 123 route files total at d03bed9.

Two disciplines keep this surface honest. First, the kind ledgers: the job dispatch enumerates its kinds in more than one place, and the S88 CI failure taught that adding a job kind must move every ledger in the same slice - S94 did exactly that, 23 to 24 kinds in both. Second, the route battery: scripts/run-route-battery.mjs plus the per-stage route probes exercise the surface end to end in CI, so an endpoint that loses its scope or its gate fails a check rather than a tenant.

7. AI Providers and Intelligent Engines

7.1 The provider layer

Eleven files under src/lib/tubeforge/providers wrap every external AI capability - chat, text-to-speech, image, video, ASR and vision - behind one registry with shared HTTP handling, configuration and per-user settings. The zai-video-core module carries the video gateway core. Because every call goes through the same seam, cross-cutting law (spend gates, SSRF guarding, usage recording) is written once.

File	Responsibility
registry.ts	Provider registry; the single enumeration every consumer reads.
config.ts / settings.ts	Provider configuration and per-user provider settings.
http.ts	Shared HTTP client; timeouts, retries, byte caps.
chat.ts	LLM chat completion (script, director, structure passes).
tts.ts / asr.ts	Voice synthesis and speech-to-text.
image.ts / vision.ts	Image generation and VLM judging.
video.ts / zai-video-core.ts	Video generation and the gateway core.

Table 7 - The 11-file provider layer under src/lib/tubeforge/providers.

7.2 Provider selection intelligence (S90)

provider-score.ts is a pure, advisory scoring engine. The production-calibrated weights are quality 30, reliability 25, cost 20, latency 10 and fit 15 - user-overridable through the ai.scoreWeights setting, tolerantly parsed, clamped, and all-zero-safe. The engine blends the organization's own UsageRecords into the priors Bayesian-style: success rate and mean latency score the reliability and latency factors, and failures count toward both reliability and the cost average. Two laws are pinned in tests: the unknown-pricing law - a paid provider with an unpriced model scores at the conservative fallback, never free, because free must be established by design (the gateway, self-hosting, or a no-key provider) - and the advisory-only law: dispatch reads the user's configured provider; the score explains, it never chooses.

Band (words per minute)	Verdict	Role targets (S92)
Under 130	Dragging	Hook: 185 wpm
130 - 175	Measured	Body: 155 wpm
175 - 205	Brisk	Close: 150 wpm
Over 205	Rushed	A brisk hook fits its role

Table 8 - The narration pacing law (S92): bands measured from real scene audio, targets per beat role.

7.3 The 2.0 intelligent systems

The TubeForge 2.0 track (ROADMAP items 4 through 13) layered intelligence onto the shared systems rather than forking pipelines. All thirteen items are landed at v1; the governing rule throughout was "shared contracts first, no parallel pipelines."

System	Session	What v1 delivers
Visual Consistency Engine	S77-S83	Unified style engine, appearance dimension, repaint re-checks, shot-art locks, cross-video character identity, pose-level rows.
Video Critic & Repair	S84-S86	Rendered-cut quality report (ship / hold / insufficient-evidence), the critic card, one-click repair firing the named affordance.
Reference intelligence	S88	Upload-only ingestion, scene map + keyframes + ASR + VLM + LLM passes (fail-open), the copyright wall: structure to the prompt, never the transcript.
Stock intelligence	S89	Pexels + Pixabay adapters, license-floored ranker, sequential frame verifier with early exit, the never-silent verification law.
Provider selection	S90	The advisory scoring engine of Section 7.2.
Production board	S91	One snapshot endpoint over existing tables: five stages, five approval checkpoints, 30-day costs, live polling.
Narration pacing	S92	Measured scene WPM from real audio durations; English bands and role targets; advisory strip, never auto-resynthesis.
Character rig	S93	Breath-cycle motion plans in the zoompan vocabulary, amplitude capped at 3%, stillness contract senior to the rig.
3D engine	S94	three.js under SwiftShader through the whiteboard pipeline; the clip rides as an ordinary video-kind asset; Blender recorded as the offline exporter.

Table 9 - The TubeForge 2.0 intelligent systems (all at v1).

8. Testing and Quality Assurance

8.1 The unit suite

The unit suite runs through scripts/test-unit.mts and stands at 471 passing, zero failing, at d03bed9. It grew by 27 tests across the last six ROADMAP sessions alone (Figure 2), because the standing practice is that every v1 lands with its contract pinned: the S89 stock work alone added 16 pins, and the S88 doc laws, copyright wall and pacing bands are all unit-pinned deterministically. Two further suites run separately and gate the same commit: the billing suite (scripts/test-billing.mts) and the queue suite; npm run check chains typecheck, lint, the unit suite and the billing suite into the one command a maintainer runs before pushing.

471 / 471

Unit tests passing at d03bed9 - plus the billing and queue suites, 7 Playwright e2e specifications, and the 18-check S87 acceptance battery.

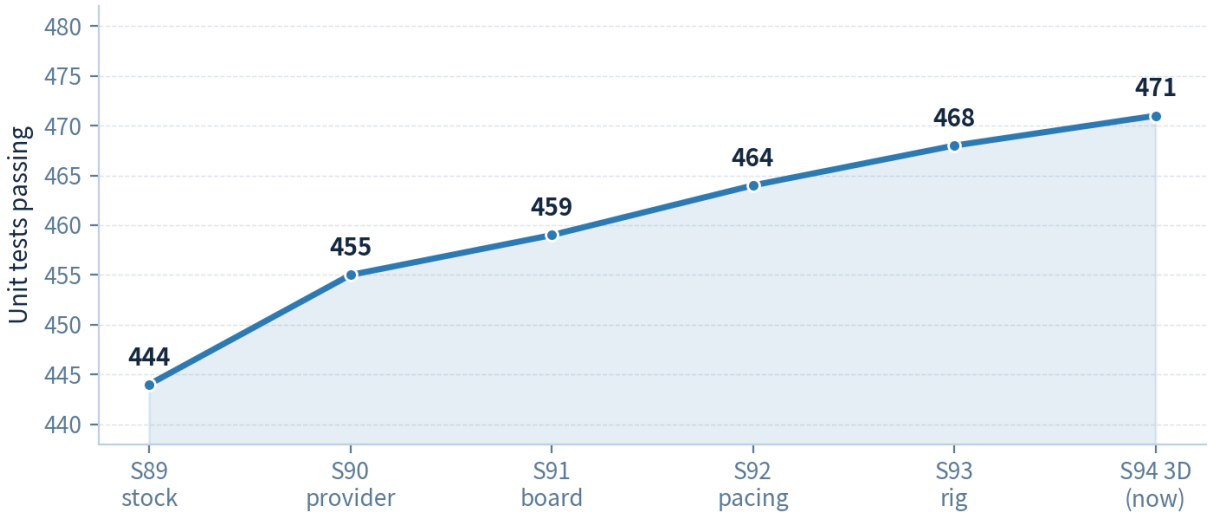


Figure 2 - Unit-test count across the closing 2.0 sessions, from the worklog receipts (S89 through S94).

8.2 End-to-end and acceptance

Specification (e2e/)	What it pins
smoke.spec.ts	The golden path: boot, auth, project creation.
isolation.spec.ts	Cross-tenant isolation end to end.
pipeline.spec.ts	Script to rendered cut through the queue.
premium-ux.spec.ts	The premium surfaces and their affordances.
mobile-sweep.spec.ts	The view sweep on mobile viewports.
whiteboard.spec.ts	The whiteboard lane, documents to render.
creator-intelligence.spec.ts	The 2.0 intelligence surfaces in combination.

Table 10 - The seven Playwright e2e specifications.

On top of the specs sits the S87 acceptance battery - 18 mechanical checks running real renders through both engine lanes with zero AI spend, wired into the CI e2e job. Quality enforcement beyond the suites is itself a feature area: QC enforcement landed in S40, the per-org content policy in S49, and retention reporting in S72. The nightly battery job exercises the fuller route battery so drift is caught within a day, not at release time.

9. CI/CD

9.1 The pipeline

Job (ci.yml)	What it does
secrets	Gitleaks secret scan of every push.
check	Typecheck, lint, unit suite, billing suite.
build	The production standalone build.
coverage	Coverage measurement over the unit suite.
postgres	Migration integrity against a real Postgres.
e2e	Playwright suite plus the S87 acceptance battery.

Table 11 - The six ci.yml jobs; the nightly workflow adds the battery job.

All six jobs are green on origin/main at ce7e9c3. The nightly workflow (nightly.yml) runs the battery job - the extended route battery - on a schedule so slow regressions surface within a day. The one live caveat: local HEAD d03bed9 (plus its backup commit) is two commits ahead and unpushed, because the development sandbox holds no TF_TOKEN; pushing from an authenticated environment is the first item of the runbook in Chapter 11.

9.2 The apt mirror outage (a CI lesson in three acts)

The e2e job installs ffmpeg through apt, and for several days the runner's azure mirror flapped on DNS. Attempt one appended the canonical archive to the mirror list - and failed, because apt's rotation semantics kept retrying the dead host and burned six of eight step-minutes on timeouts. Attempt two swapped the mirror with sed - better, but it depended on the swap landing before the flap. The standing fix is probe-and-swap: before apt runs, the workflow health-probes the four indexes apt update actually fetches (noble, noble-updates, noble-backports, noble-security - the per-index flap is why a single-index probe is insufficient) and swaps /etc/apt/apt-mirrors.txt wholesale to the canonical archive on any failure. Healthy days pay about three seconds of probing; outage days pay at worst a sixty-second probe and never touch the dead mirror.

The same incident exposed a second trap: with the mirror briefly recovered, the probe passed and the step still hung - systemd-run was waiting indefinitely on apt-daily.service, whose unattended-upgrades run had been stuck since boot. The apt lock wait has no timeout of its own, so the workflow now bounds it (timeout 60 on the apt-daily wait, DPkg::Lock::Timeout=120 on apt itself) and the step budget went from 8 to 10 minutes for headroom. The general laws, both now in the workflow comments: never depend on apt's rotation semantics, and every lock wait needs a timeout you own.

10. Deployment and Operations

10.1 The topology of record

The S33 deployment decision fixed one topology. Four artifacts had accumulated - fly.toml, the Dockerfile, docker-compose.yml and vercel.json - and the decision recorded which is real: Fly.io runs the app, with the web process group (two machines behind the Fly proxy) and the worker process group (one machine, scale with fly scale count worker=2) built from one image. The deciding constraint is the worker: rendering and the durable queue are long-running processes a serverless platform cannot host, which is also why Vercel is marked preview-only and carries no data.

Layer	Choice of record
Application	Fly.io: web (2 machines) + worker (1, scalable) from one image.
Database	Neon Postgres (or equivalent managed Postgres); prisma migrate deploy.
Object storage	Cloudflare R2 (S3-compatible) with presigned-URL delivery; local-disk mode for single-box dev.
Previews	vercel.json - fluid-compute preview builds only; NOT production.
Local development	docker-compose.yml: Postgres + web + worker.
Voice server (optional)	Self-hosted Chatterbox Turbo on a persistent box (Oracle free-tier ARM works); TUBEFORGE_SHARED_VOICE_URL/KEY.

Table 12 - The deployment topology of record (S33 decision).

10.2 Provisioning and migration procedure

First deploy: create the Neon project and the R2 bucket, then fly launch --no-deploy --copy-config, set the secrets (at minimum TUBEFORGE_AUTH_SECRET, TUBEFORGE_ENCRYPTION_KEY, DATABASE_URL and the S3 block), and deploy. Migrations run explicitly on first deploy - the postinstall script also baselines an unmanaged database automatically, but the explicit form is the procedure of record:

Schema and deploy

```
npx prisma migrate deploy --schema prisma/schema.prisma
fly deploy                                        # web + worker from one image
fly scale count worker=2 --app           # scale the worker tier
```

10.3 Backups and the restore drill

Two scripts cover backups: scripts/backup.sh for the local dev stack and scripts/backup-postgres.sh (npm run db:backup:pg) for the production database, with the restore procedure drilled in docs/runbooks/backup-restore.md. The S33 session note stands: a backup that has never been restored is a hypothesis, not a backup - the runbook exists so the drill is a documented, repeatable path rather than an improvisation during an incident.

10.4 The outstanding production gate

Section 5.3 records it and it belongs in every operations review: the Chromium-carrying production image has never been built and smoke-tested (the dev sandbox has no Docker daemon). The Dockerfile lines are in, the dependency closure is verified statically, and a real docker build plus an in-container whiteboard render remains a required gate before the container is called production-ready. It is ROADMAP Critical item 2.

11. Documentation Audit and Maintenance Runbook

11.1 What the audit found

The audit re-measured every headline figure in the repository documents against the working tree at d03bed9. Three numbers in ARCHITECTURE.md have drifted, all with the same root cause: the document was reconciled against the live source at S75 (2026-11-10), and the S88 through S94 slices landed after that reconciliation without a fresh pass. The design contracts for those slices live in docs/design/, which is correct as a practice - but the headline counts in the architecture document must be re-derived whenever the reconciliation ages past a landing.

116 -> 123 | 22 -> 23 | 117 -> 132

Route files, migrations and library modules: documented at the S75 reconciliation versus measured at d03bed9.

Artifact	Documented	Actual at d03bed9
API route files (ARCHITECTURE.md 1.1)	116	123 route.ts files
Prisma migrations (ARCHITECTURE.md 1.2)	22	23 (s94_scene3d added)
Library modules (ARCHITECTURE.md 1.1)	117	132 under src/lib/tubeforge
Prisma models	32	32 (in agreement)
ROADMAP 2.0 items	13 planned	13 / 13 landed at v1
Worklog coverage	through S75	through S94 (THE LAST ROW)

Table 13 - Documentation drift audit, measured 2026-11-12.

11.2 The standing reconciliation practice

The fix is not a one-off edit; the S75 reconciliation record (docs/audits/s75-architecture-reconciliation-record.md) defines the practice and it should be re-run after every engine-level landing: re-derive the counts from the four sources of truth (prisma/schema.prisma, prisma/migrations/, src/app/api/ and src/lib/tubeforge/), update the headline numbers in ARCHITECTURE.md, and record the reconciliation in docs/audits/. Two companion laws from the worklog: when a job kind is added, every kind ledger moves in the same slice (the S88 lesson, applied correctly in S94); and the worklog is the session log - 67 session headers from S0 to S94, with S94 marking the last ROADMAP row.

11.3 The maintenance runbook

Command	Purpose
npm run check	Typecheck + lint + unit suite + billing suite. Run before every push.
npm test	The unit suite alone (471 tests at d03bed9).
npm run test:e2e	The seven Playwright specifications.
npm run worker	Run the durable-queue worker locally.
npm run db:migrate	Create a migration from schema changes.
npm run db:sync-sqlite	Regenerate the SQLite dev mirror from the source schema.
npm run db:backup:pg	Backup the production Postgres (see the restore runbook).
npm run auth:reset	Local credential reset helper.
node scripts/measure-boot.js	The boot-payload diagnostic added with d03bed9.
git push origin main	Push the two unpushed commits (needs TF_TOKEN; first runbook item).

Table 14 - The maintenance command set.

11.4 The standing risk register

Risk	State and discipline
Docker image smoke (Critical 2)	PENDING - needs a Docker-capable runner; the gate before calling the container production-ready.
S74 real motion measurement (Critical 1)	OPEN - the auto-resume loop (cron 521163) drives 8 paired renders + 16 blind scores.
Timeline soak gate	14+ days with zero v1-document renders before the legacy fallback retires.
Z.ai gateway outages	One-probe resumption discipline; LLM whiteboard planner stays gated on it.
Production TUBEFORGE_ENCRYPTION_KEY on Vercel	Owner-side, unverified; Vercel carries no production data but the key audit is open.
Open dependency PRs	Majors tested individually, never batch-merged.
Unpushed local commits	d03bed9 + backup commit await a TF_TOKEN-capable push.

Table 15 - Standing risks tracked in the worklog and ROADMAP.

Appendix A. Quick Reference

A.1 Environment variables

Variable	Purpose
TUBEFORGE_AUTH_SECRET	Session signing. The production fail-safe refuses to serve without it.
TUBEFORGE_ENCRYPTION_KEY	AES-256-GCM key material for provider credentials and YouTube tokens (v1. format).
DATABASE_URL	PostgreSQL connection string (Neon in production).
TUBEFORGE_STORAGE_DRIVER	local or s3 - S3/R2 mode is the default for any real deployment.
TUBEFORGE_S3_* (endpoint, bucket, keys)	The S3-compatible block; presigned-URL delivery is built and tested.
TUBEFORGE_SHARED_VOICE_URL / _KEY	The optional self-hosted voice server (Chatterbox Turbo).
TUBEFORGE_SPAWN_VOICE_SERVER	Supervisor-spawned voice server - single-box dev only.
WB_CHROMIUM_LAUNCH_ARGS	Chromium launch discipline: --no-sandbox --disable-dev-shm-usage.
PLAYWRIGHT_BROWSERS_PATH	/opt/ms-playwright in the production image.

Table 16 - The environment variables a maintainer touches.

A.2 Document index

Path	What lives there
ARCHITECTURE.md	Current (S75-reconciled) and target architecture. Headline counts re-derive on every landing (Ch. 11).
ROADMAP.md	Stages 1-7 plus the 2.0 track table - 13 items, all at v1, with tails recorded.
worklog.md	The session log: 67 headers, S0 to S94. The narrative source of record.
DEPLOYMENT.md	The topology of record, provisioning checklist, Chromium worker contract.
SECURITY.md / SETUP.md / RISK_REGISTER.md	Security and release notes; the local setup guide; the standing risk register.
docs/design/s84-s94	Design contracts for the 2.0 systems (critic, timeline, reference, stock, provider, board, pacing, rig, 3D).
docs/audits/	29 audit and acceptance records, including the S75 architecture reconciliation.
docs/baselines/	Lint, coverage and benchmark baselines per session.
docs/runbooks/	backup-restore.md and the S55 human evaluation kit.

Table 17 - Where the knowledge lives.

This handbook was generated from the live repository at revision d03bed9 on 12 November 2026. Figures were measured, not quoted: route, module and migration counts come from the working tree; test counts from the suite run; the boot payload from scripts/measure-boot.js receipts; and the session-by-session growth from the worklog records. When the repository moves, re-measure - Chapter 11 exists because documents that quote numbers age faster than the systems they describe.